

Trabajo Práctico Integrador - Backend de Aplicaciones 2025

Explicación técnica del Diseño

Este documento contiene el guión técnico de exposición para el video explicativo del diseño del TPI 2025. Incluye la estructura narrativa, los puntos técnicos clave y las partes sugeridas para cada integrante del grupo.

Parte 1 - Presentación general

TPI 2025 de Backend de Aplicaciones, vamos a mostrar el diseño técnico de nuestro sistema de logística de contenedores. El objetivo del sistema es permitir que un cliente solicite el traslado de un contenedor desde un punto de origen hasta un destino, y que el operador logístico pueda asignar camiones, rutas y depósitos intermedios. Nuestro diseño está basado en una arquitectura de microservicios, que nos permite dividir el sistema en componentes independientes pero conectados. Cada microservicio tiene su propia base de datos, su propio modelo de negocio y se comunica a través de APIs REST.”

Parte 2 - Diagrama de Contenedores (Arquitectura general)

En este diagrama de contenedores podemos ver los diferentes componentes que forman parte del sistema. Arriba del todo, tenemos a los tres tipos de usuarios: - Cliente, que crea solicitudes de transporte, - Operador, que gestiona las rutas y la flota, - y Transportista, que ejecuta los tramos del viaje.

Todas las peticiones pasan por el API Gateway, que actúa como punto de entrada y rutea las solicitudes hacia los microservicios correspondientes. El Gateway también valida los tokens JWT emitidos por Keycloak, nuestro servidor de identidad. Luego tenemos dos microservicios principales: - Orders Service, que maneja la parte administrativa: clientes, contenedores, solicitudes y tarifas. - Y el Fleet Service, que se encarga de la parte logística: camiones, depósitos, rutas y tramos.

Cada servicio tiene su propia base de datos PostgreSQL independiente, lo que permite mantener bajo acoplamiento y evitar conflictos en los esquemas. Además, el Fleet Service se comunica con la API de Google Maps Distance Matrix para obtener distancias y tiempos estimados entre los puntos de origen, depósito y destino.

Parte 3 - Diagrama Entidad-Relación (DER)

Acá podemos ver el modelo de datos del sistema dividido en dos bloques: uno para cada microservicio. Del lado del Orders Service, tenemos las entidades Cliente, Contenedor, Solicitud y Tarifa. Del lado del Fleet Service, tenemos Camión, Depósito, Ruta y Tramo. En la versión final del DER también incluimos la relación entre Tramo y Depósito, para reflejar que un tramo puede tener un depósito de origen o de destino. Esto hace que el modelo sea más fiel a la realidad del negocio, donde los contenedores pueden pasar por uno o más depósitos antes de llegar al destino final.

Finalmente, noten que la entidad Ruta tiene el campo solicitudId, que conecta el Fleet Service con el Orders Service. Esa relación no es una foreign key real, sino una referencia lógica entre servicios. Esto mantiene la independencia entre bases de datos, respetando los principios de microservicios.

Parte 4 - Documentación de Recursos y Endpoints

En el Orders Service tenemos endpoints como `/api/clientes`, `/api/contenedores`, `/api/solicitudes` y `/api/tarifas`, cada uno con roles y permisos distintos. Los clientes pueden crear solicitudes y ver su estado, los operadores pueden administrar las solicitudes y contenedores, y los administradores configuran las tarifas.

En el Fleet Service, se gestionan camiones, depósitos, rutas y tramos. Los operadores pueden crear rutas, los transportistas marcan inicio y fin de tramo, y los administradores registran los camiones y depósitos.

Cada endpoint está protegido por Keycloak, y solo se puede acceder con un token JWT válido. También preparamos ejemplos JSON y documentación Swagger para cada endpoint.

Parte 5 - Cierre

En resumen, nuestro diseño sigue una arquitectura distribuida, modular y segura. Cada microservicio cumple una función específica y se comunica mediante APIs REST. Keycloak se encarga de la seguridad centralizada y la separación de roles, y la integración con Google Maps nos permite obtener estimaciones reales de distancia y costo.

Con este diseño buscamos cumplir los objetivos del TPI, aplicando los conceptos de microservicios, autenticación OAuth2, y persistencia distribuida con bases de datos independientes.